

소프트웨어 엔지니어링과 LLM 평가

코딩 벤치마크의 변천사

Curt Park

Agenda

1. LLM & 소프트웨어 엔지니어링
2. HumanEval: LLM 코딩 벤치마크의 조상님
3. SWE-bench: 실전 난이도의 문제
4. SWE-bench-Verified: 더 나은 품질
5. Multi-SWE-bench: 다언어 확장
6. The SWE-Bench illusion: 오염 문제
7. SWE-rebench: 오염 문제 대응 전략1 - 지속적 갱신
8. SWE-bench Pro: 오염 문제 대응 전략2 - 접근 제한
9. 남은 과제
10. 의견 나누기

오늘 발표에서

다루는 것

- SWE 분야에서의 LLM 활용
- SWE 벤치마크의 계보
- SWE 벤치마크의 한계

다루지 않는 것

- 모델 성능 개선
- 에이전트 하네스 (스캐폴딩)

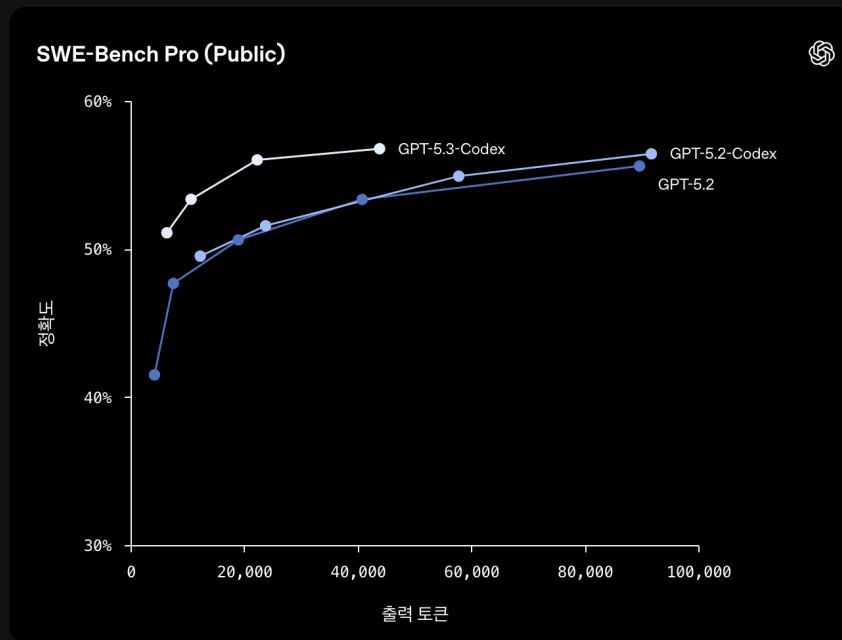
LLM & 소프트웨어 엔지니어링

최신 모델 발표와 벤치마크

최신 모델의 코딩 성능비교에 반드시 등장하는 SWE 계열의 벤치마크

	Opus 4.6	Opus 4.5	Sonnet 4.5	Gemini 3 Pro	GPT-5.2 (all models)
Agentic terminal coding Terminal Bench 2.0	65.4%	59.8%	51.0%	56.2% (34.2% self-reported)	64.7% (34% self-reported) (CodeCL)
Agentic coding SWE-bench Verified	80.8%	80.9%	77.2%	76.2%	80.0%
Agentic computer use OSWorld	72.7%	66.3%	61.4%	—	—
Agentic tool use 12-bench	91.9% 99.3%	88.9% 98.2%	86.2% 98.0%	85.3% 98.0%	82.0% 98.7%
Scaled tool use MCP Atlas	59.5%	62.3%	43.8%	54.1%	60.6%
Agentic search BrowseComp	84.0%	67.8%	43.9%	59.2% (Deep Research)	77.9% (Pro)
Multidisciplinary reasoning HuggingFace Exam	40.0% 53.1%	30.8% 43.4%	17.7% 33.6%	37.5% 45.8%	36.6% 50.0%
Agentic financial analysis Finance Agent	60.7%	55.9%	54.2%	44.1%	56.6% (S)
Office tasks GDPv3-AA Eto	1606	1416	1277	1195	1462
Novel problem-solving ARC AGI 2	68.8%	37.6%	13.6%	45.1% (Deep Thinking)	54.2% (Pro)
Graduate-level reasoning GPQA Diamond	91.3%	87.0%	83.4%	91.9%	93.2% (Pro)
Visual reasoning MMMU Pro	73.9% 77.3%	70.6% 73.9%	63.4% 68.9%	81.0% —	79.5% 80.4%
Multilingual Q&A MMMLU	91.1%	90.8%	89.5%	91.8%	89.6%

Anthropic — Claude Opus 4.6 발표 (2025)



OpenAI — GPT-5.3-Codex 발표 (2025)

"Coding = Software Engineering?"

대표적인 Agentic Coding 벤치마크들...

- SWE-bench
- SWE-bench Verified
- Multi-SWE-bench
- SWE-bench Pro
- SWE-rebench

그렇다면...

코딩(Coding)이 곧
소프트웨어 엔지니어링(SWE)일까?

→ 소프트웨어 엔지니어링이 무엇인지 먼저 짚고 넘어가보자

소프트웨어 엔지니어링이란?

와 많다!

V3 (2014)	V4 (2024)
Introduction	Introduction
1. Software Requirements	1. Software Requirements
	2. Software Architecture
2. Software Design	3. Software Design
3. Software Construction	4. Software Construction
4. Software Testing	5. Software Testing
	6. Software Engineering Operations
5. Software Maintenance	7. Software Maintenance
6. Software Configuration Management	8. Software Configuration Management
7. Software Engineering Management	9. Software Engineering Management
8. Software Engineering Process	10. Software Engineering Process
9. Software Engineering Models and Methods	11. Software Engineering Models and Methods
10. Software Quality	12. Software Quality
	13. Software Security
11. Software Engineering Professional Practice	14. Software Engineering Professional Practice
12. Software Engineering Economics	15. Software Engineering Economics
13. Computing Foundations	16. Computing Foundations
14. Mathematical Foundations	17. Mathematical Foundations
15. Engineering Foundations	18. Engineering Foundations
Appendix A. Knowledge Area Specifications	Appendix A. Knowledge Area Specifications
Appendix B. Standards	Appendix B. Standards
Appendix C. Consolidated Reference List	Appendix C. Consolidated Reference List

Bourque & Fairley (eds.), SWEBOK V4, IEEE, 2024

소프트웨어 구현

소프트웨어 테스트

소프트웨어 유지보수

소프트웨어 보안

요구사항 공학

소프트웨어 설계

소프트웨어 아키텍처

형상 관리

소프트웨어 품질

SE 운영

SE 관리

SE 프로세스

SE 모델/방법론

SE 전문실무

SE 경제학

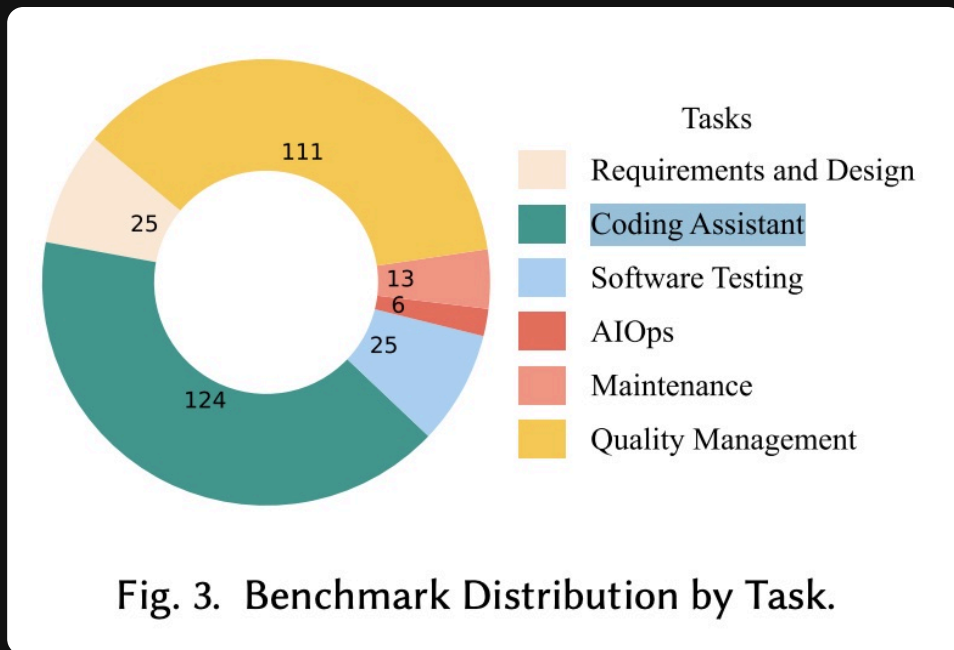
컴퓨팅 기초

수학 기초

공학 기초

소프트웨어 엔지니어링 벤치마크 분포

소프트웨어 엔지니어링 벤치마크 291개를 분석해보니, 무려 43%가 코딩!

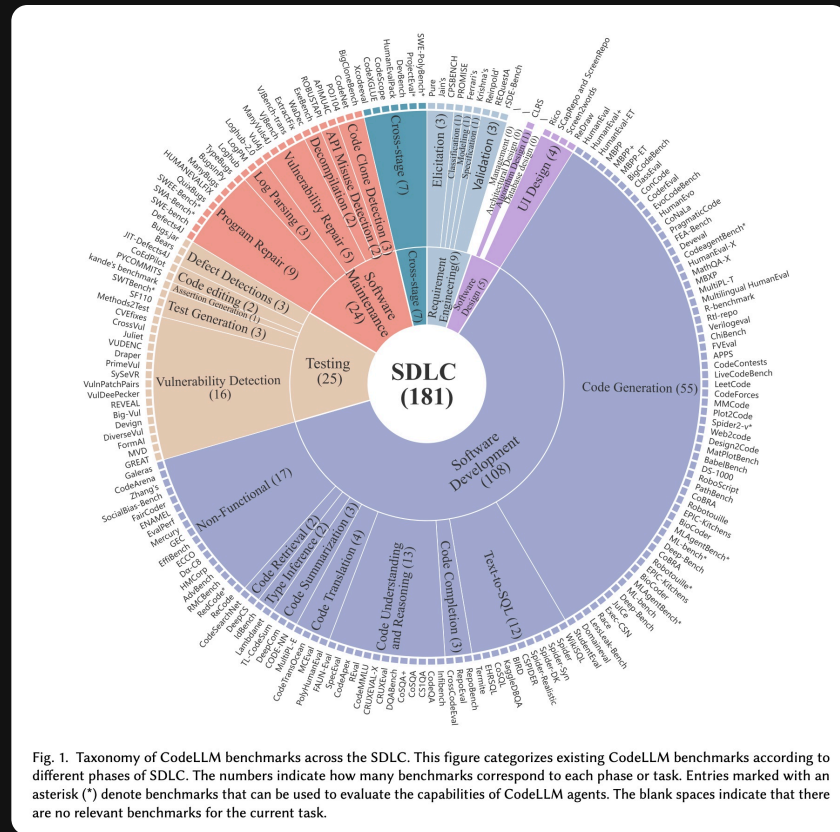


Source: Hu et al., "Assessing and Advancing Benchmarks for Evaluating LLMs in SE Tasks," arXiv:2505.08903, 2025

왜 이렇게 치우쳐져 있을까?

정량 평가 & 자동화의 난이도

SWE 영역	정답 판별 방법	자동화
코딩	테스트 실행	✅ 쉬움
테스팅	커버리지/버그 탐지	⚠️ 부분
요구사항	전문가 판단 필요	❌ 어려움
설계/아키텍처	주관적 기준	❌ 어려움
프로젝트 관리	성과 지표 다양	❌ 어려움



즉, 상대적으로 평가가 용이한 특정 주제로
쏟림 현상이 발생하는 것

오늘 소개해드릴 내용은,
소프트웨어 엔지니어링의 극히 일부분 영역!

HumanEval

LLM 코딩 벤치마크의 조상님 (2021)

배경: BLEU 점수의 한계

기존 평가의 문제

- 코드 평가에 **BLEU** 점수 사용
- **BLEU** (Bilingual Evaluation Understudy):
n-gram 중복 기반 텍스트 유사도 메트릭.
기계번역 평가용으로 설계됨.
- 실제로 동일한 기능이 실행되는지를 확인하기는 어려움

왜 BLEU로는 부족한가?

```
# 정답
def add(a, b): return a + b

# BLEU 높지만 틀린 코드
def add(a, b): return a - b

# BLEU 낮지만 맞는 코드
def sum_two(x, y): return x + y
```

→ 기능적 정확성(functional correctness)을 직접 측정해야 한다

HumanEval이란?

문제 예시

```
def has_close_elements(
    numbers: List[float],
    threshold: float
) → bool:
    """ Check if in given list of numbers,
    are any two numbers closer to each other
    than given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0], 0.3)
    True
    """
```

평가 메트릭: pass@k

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

- n: 문제당 총 생성 샘플 수 (본 논문에서 n=200)
- c: 단위 테스트를 통과한 정답 샘플 수
- k: 평가 시 선택하는 샘플 수 ($k \leq 100$)

즉, n번 시도 중 k개를 뽑았을 때 하나라도 정답일 확률
(전부 오답일 확률을 1에서 뺀 불편 추정량)

-
- **pass@1**: 1개 생성해서 맞출 확률
→ 실용적 지표 (사용자는 보통 첫 응답을 씀)
 - **pass@100**: 100개 중 하나라도 맞을 확률
→ 모델의 잠재력 상한선

HumanEval 평가 데이터 특징

GitHub 소스코드로 학습하는 과정에서 **Codeforces** 등에 출제된 문제의 솔루션을 이미 학습했을 가능성이 있다.

→ 평가 데이터는 수작업으로 작성될 필요가 있다

164

수작업 프로그래밍 문제

4가지

구성 요소

함수 시그니처 · Docstring

함수 본문 · 단위 테스트

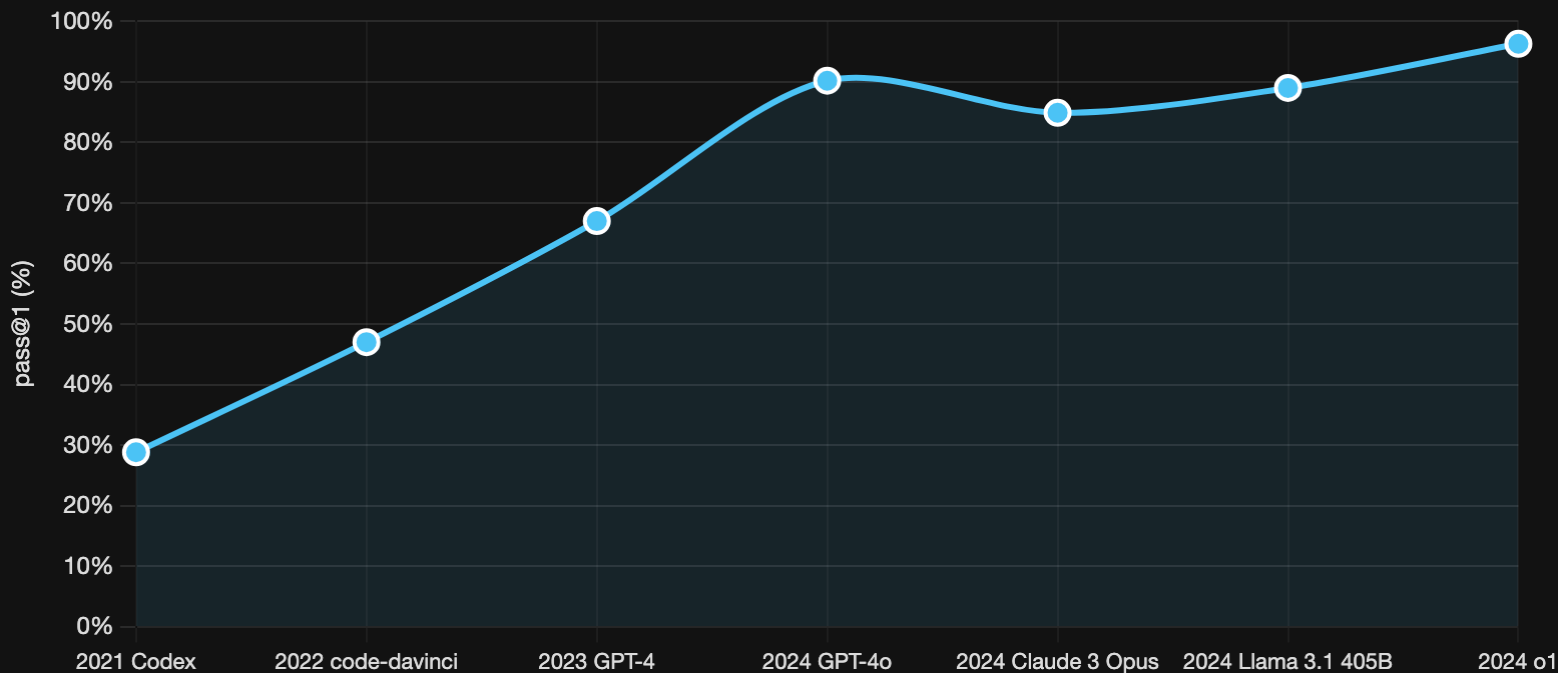
7.7

문제당

평균 단위 테스트 수

HumanEval 성능 추이

pass@1 (%) — 4년간의 진화



Codex 28.8%: Chen et al. 2021 · code-davinci 47.0%: CodeT (Chen et al. 2022) · GPT-4 67.0%: OpenAI Technical Report 2023 · GPT-4o 90.2%: OpenAI simple-evals · Claude 3 Opus 84.9%: Anthropic Model Card 2024 · Llama 3.1 89.0%: Meta AI Blog 2024 · o1 96.3%: EvalPlus Leaderboard / o1 System Card 2024

벤치마크 포화의 증거

변형 태스크에서의 성능 하락

Top 모델들이 HumanEval 원본에서는 90%+ 달성하지만, 변형된 태스크에서는 **19.6 ~ 47.7%p** 하락

EvoEval의 7가지 변형 예시:

- 함수 이름 변경 (e.g., `has_close_elements` → `check_proximity`)
- 입출력 형식 뒤집기 (e.g., `bool` 반환 → 조건을 만족하는 쌍의 개수 반환)
- 함수 합성 (e.g., `has_close_elements` + `mean` → "평균과 가장 가까운 원소 쌍 존재 여부")
- 맥락·도메인 교체 (e.g., 숫자 근접도 → 문자열 편집 거리 근접도)
- 난이도 상향 (e.g., 옛지 케이스 추가: 빈 리스트, 음수, 중복값 처리 필요)
- 미묘한 의미 변환 (e.g., `≤ threshold` → `< threshold` , 경계값 조건 변경)
- 외부 도구 활용 (e.g., `datetime` 라이브러리로 날짜 계산 추가)

→ 문제의 본질은 같지만 표현만 바뀌었을 때 풀지 못한다

→ 진짜 이해가 아니라 패턴 암기에 의존한다는 증거

HumanEval의 한계

실제 소프트웨어 엔지니어링 문제는 훨씬 더 복잡하다

HumanEval의 제약조건

- 독립적인 함수 한 개
- 명확한 입출력 명세
- 빈 컨텍스트에서 시작

현실 세계의 소프트웨어 개발

- 여러 파일, 모듈, 클래스가 얹힌 대규모 코드베이스
- 기존 구현·테스트·문서 등 프로젝트 전체 맥락 이해 필요
- GitHub 이슈를 읽고 버그를 재현하고, 기존 테스트 스위트 통과시켜야 함

→ 단일 함수 작성 능력 ≠ 소프트웨어 엔지니어링 능력

SWE-bench: 실전 난이도의 문제

실제 GitHub 이슈 기반 평가 (2023)

SWE-bench란?

Jimenez et al., Princeton NLP — ICLR 2024 Oral, arXiv:2310.06770

핵심 아이디어

유명 GitHub 레포에 제출된
이슈(버그 리포트 / 기능 요청) 해결이 목표

GitHub PR = 자연발생적 벤치마크
이슈 → 코드 수정 → 테스트 통과의
흐름이 이미 존재한다

규모와 성능

12

인기 Python
오픈소스 레포

2,294

이슈-PR 쌍

BM25 retriever + Claude 2로 단 1.96%만 해결!

HumanEval 84% 달성 모델이 실전 코드베이스 앞에서 사실상 무력화

SWE-bench 구축: 3단계 파이프라인



Figure 2: SWE-bench task instances are created from merged pull requests that resolve an issue, contributes tests, and install successfully.

Stage 1: PR 수집

- 12개 인기 레포에서
- ~90,000 PR 수집

Stage 2: 조건 기반 필터링

- GitHub 이슈와 연결된 PR
- 테스트 파일 수정 포함

Stage 3: 실행 기반 필터링

- 패치 적용 전 테스트 실패
- 패치 적용 후 테스트 통과
- 환경 오류 없음

~90,000 PR → 최종 2,294개 인스턴스 (약 2.5% 생존)

12개 레포: Django(850), sympy(386), scikit-learn(229), sphinx(187), matplotlib(184) 등

SWE-bench 태스크와 평가 방법

태스크 구성

- **Input:** 이슈 텍스트 + 코드베이스
(키워드 기반 검색으로 관련 파일을 찾아 컨텍스트 구성)
- **Output:** 패치 파일 (diff 형식)

평가 기준

1. 패치 적용
2. 새 테스트 통과 (이슈가 제기한 버그 기능 검증)
3. 기존 테스트 유지 (기존 기능 회귀 없음)
4. 둘 다 통과 → "resolved"

→ fail-to-pass + pass-to-pass 동시 충족

A.4 EVALUATION PROCEDURE

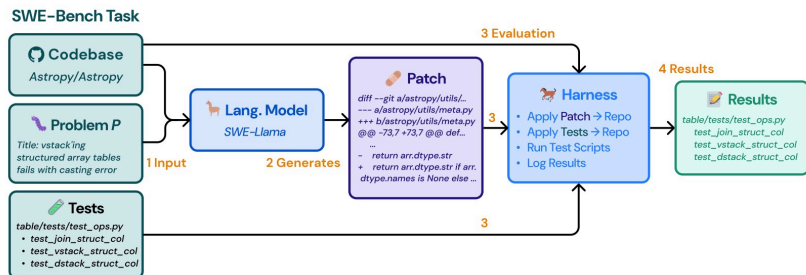


Figure 8: Visualization of the evaluation pipeline at an individual task instance level. During evaluation, the `Patch` is model generated. A prediction `.patch` must be applied successfully and produce the same results as the corresponding task instance's `D` for task completion.

$$\text{resolve rate} = \frac{\text{해결된 인스턴스}}{\text{전체 인스턴스}}$$

인스턴스당 1회 시도(pass@1) — 생성한 패치 하나로 모든 테스트를 통과해
야 resolved

SWE-bench 데이터 특성

패치 통계

- 평균 1.7개 파일 수정
- 평균 3.0개 함수 수정
- 평균 32.8줄 변경
- 이슈 설명 평균 195단어
- pass-to-pass 테스트 중간값 51개

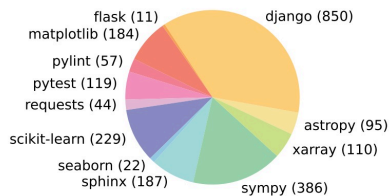


Figure 3: Distribution of SWE-bench tasks (in parenthesis) across 12 open source GitHub repositories that each contains the source code for a popular, widely downloaded PyPI package.

Table 1: Average and maximum numbers characterizing different attributes of a SWE-bench task instance. Statistics are micro-averages calculated without grouping by repository.

		Mean	Max
Issue Text	Length (Words)	195.1	4477
Codebase	# Files (non-test)	3,010	5,890
	# Lines (non-test)	438K	886K
Gold Patch	# Lines edited	32.8	5888
	# Files edited	1.7	31
	# Func. edited	3	36
Tests	# Fail to Pass	9.1	1633
	# Total	120.8	9459

레포별 인스턴스 분포

SWE-bench 초기 결과

1.96%

Claude 2
BM25 13k

~5%

Oracle retrieval
(실제 PR이 수정한 파일을 컨텍스트로 제공)

HumanEval 84% 달성 모델이 SWE-bench에서는 2% 미만

컨텍스트 구성 방식이 성능을 좌우한다

Table 3: BM25 recall with respect to oracle files for different maximum context lengths.

	BM25 Recall		
	13k	27k	50k
Avg.	29.58	44.41	51.06
All	26.09	39.83	45.90
Any	34.77	51.27	58.38

Table 3: BM25 Recall

컨텍스트 길이↑ → recall↑

Table 5: We compare models against each other using the BM25 retriever as described in Section 4.

Model	SWE-bench		SWE-bench Lite	
	% Resolved	% Apply	% Resolved	% Apply
Claude 3 Opus	3.79	46.56	4.33	51.67
Claude 2	1.97	43.07	3.00	33.00
ChatGPT-3.5	0.17	26.33	0.33	10.00
GPT-4-turbo	1.31	26.90	2.67	29.67
SWE-Llama 7b	0.70	51.74	1.33	38.00
SWE-Llama 13b	0.70	53.62	1.00	38.00

Table 5: BM25 검색 시 해결률

Claude 2 1.96%

Model	"Oracle" Retrieval	
	% Resolved	% Apply
Claude 2	4.80	62.82
ChatGPT-3.5	0.52	21.80
GPT-4*	1.74	34.00
SWE-Llama 7b	3.01	65.52
SWE-Llama 13b	3.97	66.78

Table 18: We compare models against each other using the oracle retrieval settings as described in Section 4. The main results table, Table 5, presents the results for the different models when using BM25 only. *Due to budget constraints we evaluate GPT-4 on a 25% random subset of SWE-bench in the "Oracle" and BM25 27K retriever settings only.

Table 18: Oracle 검색 시 해결률

Claude 2 4.80%

recall↑에도 해결률은 오히려 하락 — 파일을 더 많이 넣는다고 해결되지 않는다

Oracle로 정확한 파일을 알려줘도 최대 4.8% 수준

수정 파일 외의 맥락(의존 모듈·테스트·히스토리)도 이슈 해결에 필요
→ 동적으로 탐색하는 에이전트 스캐폴딩 연구로 이어짐 (오늘 발표 범위 밖)

SWE-bench의 품질 문제

2,294개 중 상당수가 모호하거나 불명확 → 올바른 솔루션도 테스트 실패 가능

실제로 발견된 문제들:

① 불완전한 이슈 설명 — PR 토론까지 읽어야 이해 가능

e.g. "Fix the regression from #1234" — #1234를 모르면 무엇을 고쳐야 할지 알 수 없음

② 과도하게 구체적인 테스트 — 로직은 맞아도 표현이 다르면 실패

e.g. 에러 메시지가 `"invalid input"` 대신 `"Input is invalid"` 이면 테스트 실패

③ 이슈 본문에 없는 배경 지식 필요

e.g. Python 3.9에서 deprecated된 `collections.Callable` → `collections.abc.Callable` 변경이 필요한데, 이슈엔 언급 없음

→ 품질 검증된 부분집합이 필요하다

SWE-bench Verified: 더 나은 품질

전문가 검증 부분집합 (2024)

왜 Verified가 필요했나

OpenAI Preparedness Team, 2024

Overly Specific Tests

테스트가 너무 구체적이거나
이슈와 무관한 내용을 검사

예: 특정 에러 메시지 문자열 일치 검사 → 논리는 맞지만 메시지가 다르면 실패

Underspecified Issues

이슈 설명이 모호해서
무엇을 해결해야 하는지 불명확

예: PR 토론 맥락 없이는 이슈 본문만으로 해결책을 특정할 수 없음

Environment Setup

환경 설정 문제로
올바른 솔루션도 실패

예: 테스트가 특정 패키지 버전에서만 통과하도록 작성되어 있는데, 평가 환경에 다른 버전이 설치되면 올바른 코드도 실패

원본 2,294개 중 **68.3%**가 위 문제 중 하나를 가짐

→ 올바른 코드를 작성해도 테스트를 통과 못하는 케이스가 다수 존재

→ 모델이 실제로 이슈를 해결했음에도 "미해결"로 집계 → 해결률이 실제 능력보다 낮게 측정

SWE-bench Verified: 검증 방법론

93명

전문 Python 개발자
(Upwork 모집)

1,699개

수동 평가 샘플

×3

인스턴스당
독립 평가 횟수

심각도 스케일 (0~3점)

점수	의미	처리
0~1점	경미한 문제	✅ 유지
2~3점	심각한 문제	❌ 제거

보수적 양상블: 3명 중 가장 높은 점수 채택

→ 단 1명이라도 2점 이상이면 제거

2가지 평가 기준

① 이슈 명확도

이슈 설명만으로 해결책을 특정할 수 있는가?

② 테스트 공정성

FAIL_TO_PASS 테스트가 올바른 솔루션을 거부하는가?

+ 난이도 추정(Easy/Medium/Hard) · 기타 문제 자유 기재

SWE-bench Verified: 필터링 결과

원본 SWE-bench (2,294개)



→ 500개 선정

불명확한 이슈 설명 (38.3%)



부당한 테스트 케이스 (61.1%)



문제 유형

비율

불명확한 이슈 설명

38.3%

부당한 테스트 케이스

61.1%

전체 제거 비율

68.3%

최종 선정

500개

$38.3\% + 61.1\% \neq 68.3\%$

두 문제를 동시에 가진 인스턴스(≈31%)가
중복 집계되기 때문

원본 2,294개 중 **68.3%** 제거
→ 최종 **500개**만 Verified 인증

SWE-bench Verified: Docker 평가 인프라

왜 Docker가 필요했나?

Before: 팀마다 서로 다른 환경 → 결과 비교 불가
"우리 모델 23% 달성" — 같은 기준으로 측정한 건가?

After: 모든 제출이 동일한 컨테이너에서 실행
→ 공정하고 재현 가능한 비교

평가 과정

1. 컨테이너 마운트 (PR 직전 상태)
2. 에이전트 패치를 git diff로 적용
3. 지정된 테스트 스위트 전체 실행

3-Layer 이미지 아키텍처

Base Image

모든 평가 공통 의존성

Environment Images (~60개)

레포별 Python 환경

Instance Images (500개)

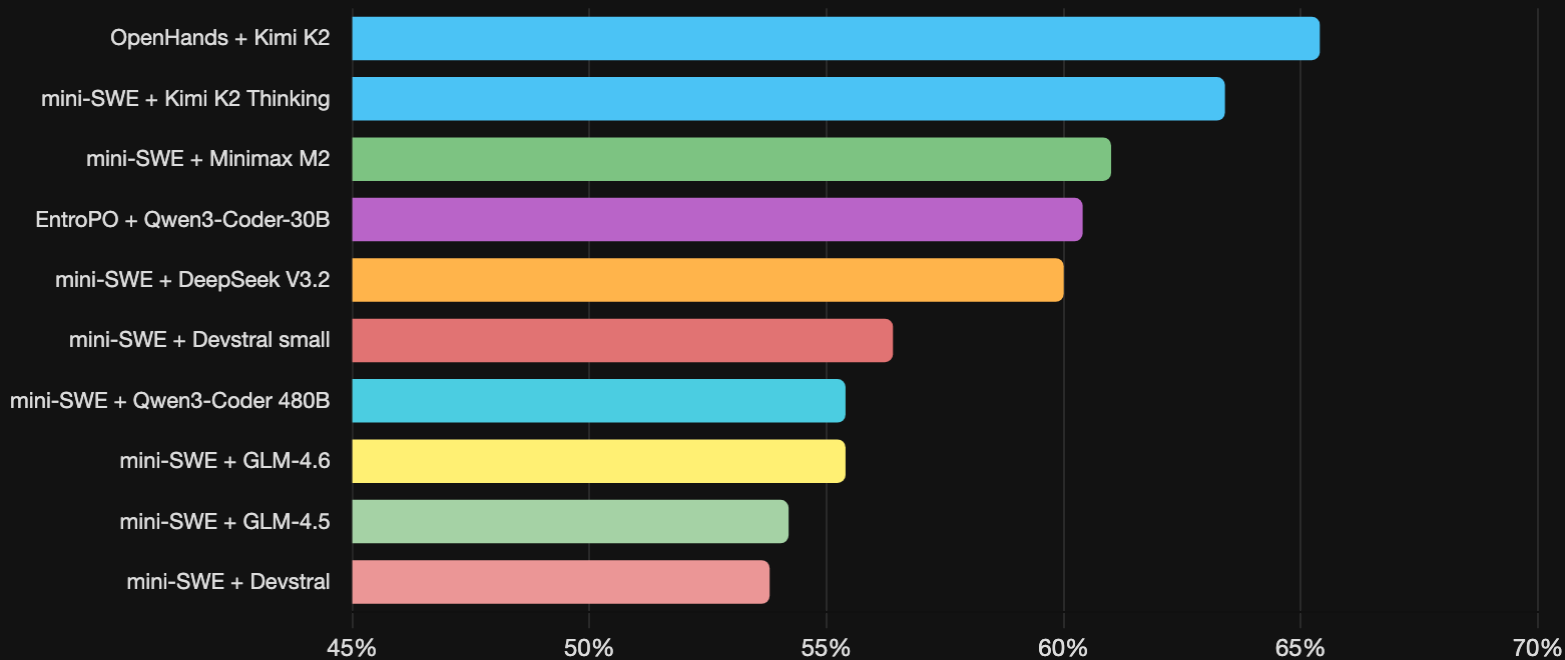
인스턴스별 정확한 코드 상태

전체 500개 이미지 = 약 30 GiB

레이어 공유로 원본(~2,000 GB) 대비 대폭 압축

SWE-bench Verified 리더보드

해결률 (%) — swe-bench.com 리더보드 Top 10 (2025년 12월 기준)



여전한 SWE-bench의 한계

Python Only

12개 레포 모두 Python
실제 개발 환경은 Java, Go,
Rust, TypeScript 등 다언어

정적 스냅샷

한번 수집 후 업데이트 없음
모든 주요 LLM의 훈련 데이터
컷오프 이전 이슈 → 오염 누적

리더보드 경쟁

점수 경쟁이 과적합을 유발
모델 훈련 시 Verified 태스크
노출 → 성능 인플레이션

→ 이 세 한계가 다음 두 질문을 낳는다:

"다른 언어에서도 통하는가?" (Multi-SWE-bench) · "점수가 진짜인가?" (The SWE-Bench Illusion)

Multi-SWE-bench: 다언어 확장

Python 편향을 넘어서 (2025)

Multi-SWE-bench: 왜 필요한가?

SWE-bench/Verified의 한계

- Python 전용 — 7개 언어의 특성 무시
- Python 특화 에이전트를 그대로 다른 언어에 적용

언어별 고유 특성

차원	예시
패러다임	OOP / 함수형 / 절차적
타입 시스템	정적(Java, Rust) / 동적(JS, Python)
메모리 관리	GC / 소유권(Rust) / 수동©
실행 모델	컴파일 / 인터프리터 / JIT

핵심 질문

Python에서 훈련·최적화된 에이전트를
7개 언어로 일반화할 수 있는가?

실제 개발 환경은 다양한 언어로 구성되어 있습니다. Python
점수로 "코딩 능력"을 일반화하는 것은 위험할 수 있다.

Multi-SWE-bench: 핵심 기여 3가지

Zan et al., ByteDance Seed — NeurIPS 2025 Datasets & Benchmarks, arXiv:2504.02605

① 다언어 벤치마크

7개 언어 × 39개 레포

1,632개 고품질 인스턴스

68명 전문 어노테이터 검증

2,456개 후보 → 엄격한 필터링

② 대규모 실증 평가

9개 LLM × 3가지 방법

GPT-4o, o1, o3-mini

Claude 3.5/3.7, DeepSeek-

V3/R1

Qwen2.5-72B, Doubao-1.5-pro

③ Multi-SWE-RL

RL 훈련용 오픈소스 데이터셋

76개 레포 / 7개 언어

4,723개 컨테이너화 인스턴스

전체 파이프라인 공개

7

지원 언어

39

레포지토리

1,632

최종 인스턴스

68

전문 어노테이터

Multi-SWE-bench: 데이터 구축 파이프라인

1 레포지토리 선정

GitHub stars 500+ · 활동기간 6개월+ · CI/CD 보유

2 PR 수집

이슈 연결 + 테스트 파일 수정 + main 브랜치 머지

3 Docker 환경 구성

CI/CD 워크플로우 · 문서에서 의존성 추출

4 실행 기반 필터링

3가지 로그(Run/Test/Fix) 테스트 상태 전이 분석

fail-to-pass 없거나 비정상 거동 → 제거

2,456개 확보

5 전문가 수동 검증

2명 독립 어노테이션 → 교차 검토

언어별 80%+ 정확도 기준 (내부 QA 14명 검증)

2,456개 → 1,632개

어노테이터 자격 기준

- 해당 언어 2년 이상 경험
- 관련 전공 학위 보유
- 1시간 사전 교육 (목표·절차·기준)
- 내부 QA팀 14명이 정답 생성 후 검증

테스트 상태 전이 기준

- ✓ 유지: FAIL → PASS (새 테스트 통과)
- ✗ 제거: ANY → PASS → FAIL 비정상 거동
- ✗ 제거: fail-to-pass 전이 없는 PR

Multi-SWE-bench: 실험 결과

방법론별 최고 성능 (해결률 %)

언어	MAgentLess	MSWE-agent	MopenHands
Python	36.2	45.8	52.2
Java	11.7	23.4	21.9
Rust	5.9	6.7	15.9
C++	7.0	11.6	14.7
Go	2.8	5.4	7.5
C	1.6	8.6	8.6

핵심 발견

Python → 타 언어 급락

MopenHands: Python 52.2% → Java 21.9% → JS 5.1%

Easy → Hard 붕괴

Easy = 단일 파일 · 짧은 패치 / Hard = 멀티파일 · 600 토큰+

Python Easy: 71.7% → Hard: 11.1%

타 언어 Hard: 대부분 0% 근접

성능 저하 트리거

패치 > 600 토큰 or 파일 2개 이상 수정 시

모든 언어·방법에서 성능 급락

The SWE-Bench Illusion

오염 문제

The SWE-Bench Illusion

Liang et al., Microsoft Research — NeurIPS 2025

핵심 질문

모델이 벤치마크를 정말 "풀고" 있는가,
아니면 "기억해내고" 있는가?

세 가지 진단 실험

설계 원칙: 추론으로는 맞출 수 없는 상황을 만들어, 모델이 맞다면 "기억"이 유일한 설명이 되도록

실험 1 — 파일 경로 맞추기

이슈 설명만 제공, 코드 접근 ×
버그 파일 경로 예측

왜 증거인가?

이슈 텍스트만으로는 "버그가

`db/sql/compiler.py` 에 있다"는 것을 추론 불가
— 코드 구조를 봐야만 알 수 있음. 맞다면 이미 알고 있었다는 뜻

실험 2 — 함수 재현

이슈 설명 + 함수를 지운 파일 제공
삭제된 함수 구현 요청
→ 5-gram 중복률 측정

왜 증거인가?

같은 기능을 구현하는 코드는 무수히 많음. 원본과 연속 5단어까지 일치한다면 추론이 아니라 텍스트를 그대로 기억해서 출력

실험 3 — 점두사 완성

수정 전 코드 앞부분만 제공
나머지 완성 요청
→ 정확 일치율 측정

왜 증거인가?

버그 수정 방법은 여러 가지가 가능. 원본 패치와 문자 단위로 정확히 일치한다면 코드를 생성한 게 아니라 재현한 것

핵심 실험: 파일 경로 맞추기

76%

SWE-bench Verified
파일 경로 정확도

53%

외부 레포지토리
파일 경로 정확도

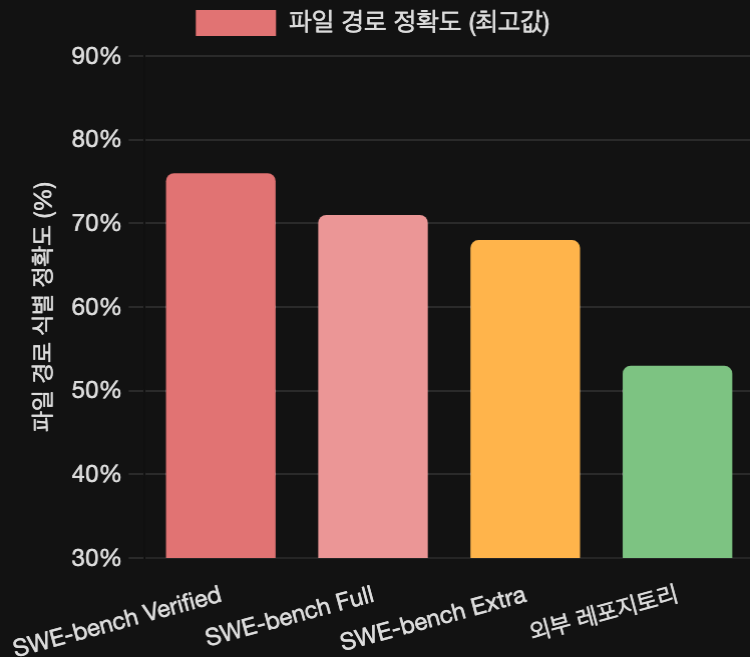
23%p

성능 격차

- 모델에게 이슈 설명만 제공 (코드 접근 불가)
- 버그가 있는 파일의 경로를 예측하도록 요청
- 코드를 보지 않고 경로를 맞힌다 → 훈련 데이터를 기억하고 있다는 의미

오염도 그래디언트: 인과 논증

파일 경로 식별 정확도 (%) — 인터넷 노출량 순



노출량이 많을수록 정확도 높음 →

데이터셋	노출 수준	정확도
SWE-bench Verified	최다 유통	76%
SWE-bench Full	많음	71%
SWE-bench Extra	적음	68%
외부 레포지토리	미포함	53%

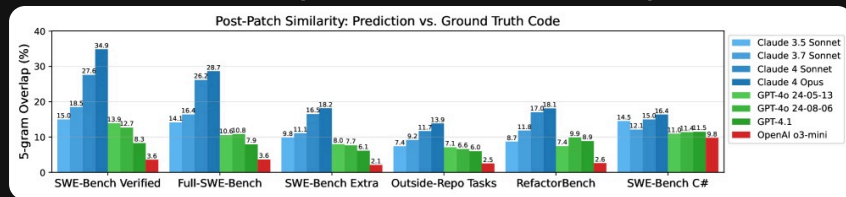
추론이었다면: 데이터셋이 달라도 정확도는 같아야 함

실제: 노출량과 정확도가 정확히 비례

→ 노출 = 암기 = 성능 → 오염의 증거

암기의 정량적 증거

함수 재현 실험 (연속 5단어 중복률)



데이터셋

최고 중복률

SWE-bench Verified

34.9%

SWE-bench Full

28.7%

외부 벤치마크

18.1%

접두사 완성 실험 (정확 일치율)

모델

정확 일치율

Claude 4 Opus

31.6%

Claude 4 Sonnet

21.4%

GPT-4o (2개 버전)

17.4~18.4%

Claude 3.7 Sonnet

12.3%

Claude 4 Opus: 코드 접두사에서
31.6%가 원본과 정확히 일치
→ 추론이 아니라 기억

시사점: 두 가지 암기 패턴과 리더보드의 함의

인스턴스별 암기

특정 이슈-솔루션 쌍을 학습 데이터에서 기억

증거: Verified > Full > Extra 단계적 성능 하락

레포 편향 암기

특정 레포의 구조와 패턴을 과적합

증거: 같은 모델이 SWE-bench 내부 레포(76%)와 처음 보는 레포(최저 29%) 간 최대 **47%p** 차이

리더보드 경쟁의 함의

- 높은 SWE-bench 점수 ≠ 높은 범용 코딩 능력
- 신규 모델일수록 더 많은 데이터에 노출
→ 점수 인플레이션

벤치마크의 가치는

오염으로부터의 자유도에 비례한다

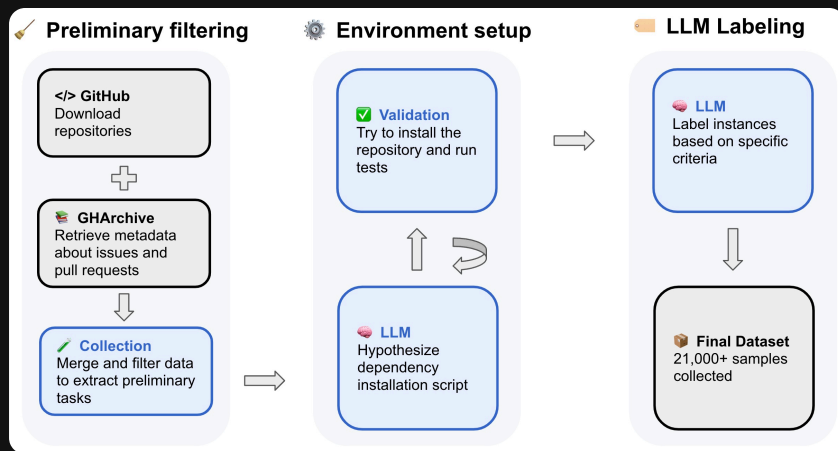
⚠️ 논문의 한계 인정: 훈련 데이터 직접 접근 불가 — 행동 패턴에서의 추론이며, "뛰어난 일반화"일 가능성을 완전 배제할 수 없음. 그러나 복수 실험·복수 모델의 일관된 그래디언트는 오염 가설을 강하게 지지

SWE-rebench: 지속적 갱신

주기적 갱신 전략 (2025)

SWE-rebench: 개요

Badertdinov et al., NeurIPS 2025 Datasets & Benchmarks, arXiv:2505.20411



핵심 수치

항목	수치
최종 인스턴스	21,336개
레포 다양성	3,468개 (vs Verified 12개)
평가용 서브셋	294개 (169개 레포)
설치 성공률	31%

핵심 전략

오염된 정적 벤치마크의 한계를
주기적 신규 태스크 추가로 우회

SWE-rebench: 4단계 자동화 파이프라인

1 예비 태스크 수집

30,000+ 레포 · Python 75%+ · 허용적 라이선스

450,000개 PR → 필터링 → **153,400개** 후보

조건: 이슈 연결 + 테스트 수정 + 1~15 파일 변경

2 설치 레시피 자동 생성

Qwen2.5-72B가 최대 3개 JSON 레시피 생성

오류 로그 피드백으로 반복 개선

레포 단위 성공률 **31%**

3 실행 기반 검증

분산 컨테이너에서 패치 전후 테스트 실행

fail-to-pass 필수 + pass-to-pass 유지

성공 시 의존성 버전 고정 → **21,336개**

4 품질 자동 평가

Qwen2.5-72B를 Verified 3,800개로 파인튜닝

3가지 이진 레이블 독립 예측 → 메타데이터 제공

품질 평가 3가지 기준

기준	정확도	F1
이슈 명확도	79%	0.76
태스크 복잡도	81%	0.82
테스트 패치 적절성	67%	0.65

하드 필터 × — 메타데이터로 제공, 사용자가 기준 직접 선택

레시피 추가해도 성능 수렴

1개→6/18, 3개→8/18, 10개→9/18

자동화만으로는 품질 향상에 한계

SWE-rebench: 오염 실증

GPT-4.1 시간대별 성능

2025년 1월 태스크 (구형)

31.1%

↓ 4.4%p 하락

2025년 3~4월 태스크 (신규)

26.7%

같은 모델 · 같은 방법 · 다른 시간대의 태스크
GPT-4.1이 유일하게 뚜렷한 하락을 보인 모델

훈련 데이터에 이미 포함된 오래된 태스크에서 성능이 높고, 훈련 이후 등장한 새 태스크에서 낮다
→ 오염이 점수를 부풀렸다는 직접 증거

전략 장단점

- ✅ 다양성 — 3,468개 레포 · ✅ 확장성 — 21,000+
- ✅ 최신성 — 지속 갱신으로 암기 불가
- ❌ 설치 성공률 31% · ❌ Python 전용

SWE-bench Pro: 접근 제한

비공개 코드 기반 평가 (2025)

SWE-bench Pro: 데이터셋 구조

Deng et al., Scale AI — arXiv:2509.16941

3단계 오염 방어 설계

Public (731개) — GPL Copyleft 라이선스

폐쇄형 모델이 GPL 코드를 학습하면 서비스 전체가 GPL 적용

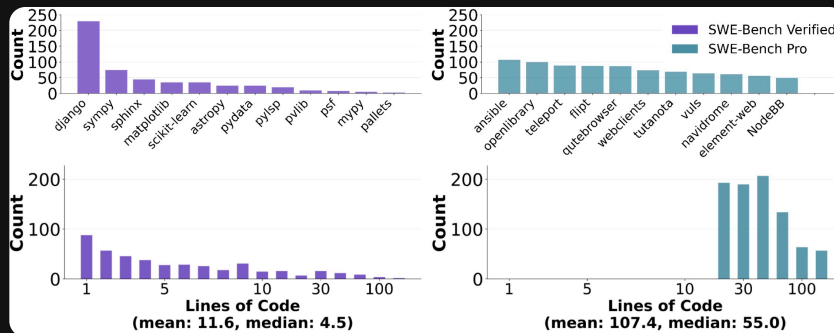
→ 법적 위험으로 학습 불가 → 자연스러운 오염 방어막

Held-out (858개) — 완전 비공개

결과만 공개, 데이터 미공개 → 미래 과적합 탐지용

Commercial (276개) — 18개 스타트업 미공개 코드

세상에 공개된 적 없음 → 훈련 데이터 포함 원천 불가



Long-Horizon 복잡도

항목	SWE-bench Pro	Verified
평균 패치 줄수	107.4줄	32.8줄
평균 수정 파일 수	4.1개	1.7개
복잡도 배율	~3배	—

SWE-bench Pro: 인간 보강

태스크 3단계 보강

자동 수집된 이슈의 8.4~25.9%는 맥락 누락 → 인간이 직접 보강

1 Problem Statement 재작성

커밋/PR에서 추출한 설명의 모호함 제거
→ 이슈만 보고도 무엇을 고쳐야 할지 명확하게

2 Requirements 명세

유닛 테스트 기반 동작 목록을 인간이 직접 작성
→ 테스트가 정말 이슈를 검증하는지 보장

3 Interface Specification

신기능 추가 시 예상 클래스/함수 시그니처 명시
→ 에이전트가 구현 방향을 잡을 수 있게

보강의 효과 — Ablation

보강 제거 시 성능 변화:

모델	보강 있음	보강 없음	하락
GPT-5	25.9%	8.4%	-17.5%p
Claude Opus 4.1	22.7%	8.2%	-14.5%p

인간 보강 없이는 성능이 1/3 수준으로 붕괴
→ 이슈 설명의 품질이 에이전트 성능의 핵심 변수

SWE-bench Pro: 성능과 실패 분석

모델별 해결률

모델	Public	Commercial
Claude Sonnet 4.5	43.6%	—
GPT-5 (high)	41.8%	15.7%
Claude Opus 4.1	—	17.8%
Gemini 2.5 Pro	—	10.1%
GPT-4o	—	3.6%

GPT-5: Public 41.8% → Commercial 15.7% (−26.1%p)

에이전트 설정: SWE-Agent · 최대 50턴 · \$2 예산

실패 원인 분석 (LLM-as-Judge)

실패 원인	Claude Opus 4.1	Claude Sonnet 4	GPT-5
잘못된 솔루션	50.3%	—	39.5%
문법 오류	31.3%	—	—
컨텍스트 오버 플로우	—	62.6%	—
무한 파일 읽 기	—	57.4%	—
도구 사용 실 패	—	—	17.7%

대응 전략 트레이드오프

	주기적 갱신	접근 제한
대표	SWE-rebench	SWE-bench Pro
장점	대규모, 자동화, 최신 이슈	오염 근본 차단 -> 품질 제어 용이
단점	개별 샘플 품질 저하	새 기술 트렌드 반영 어려움

두 전략 모두 완벽하지 않다 — 상호보완적 접근 필요

남은 과제와 전망

벤치마크 너머의 소프트웨어 엔지니어링

벤치마크 진화의 교훈

- 
- A vertical timeline with a light blue line and circular markers at each year, listing the evolution of benchmarks from 2021 to 2025.
- 2021** ● **HumanEval** — 함수 단위, Python, pass@k 도입
→ 한계: 포화 (4년 후 96%), 비현실적, 오염 취약
 - 2023** ● **SWE-bench** — 실제 GitHub 이슈, Docker 환경
→ 한계: Python 전용, 품질 문제(68% 필터링 필요)
 - 2024** ● **Verified / Multi-SWE-bench** — 품질 보장, 다언어 확장
→ 한계: 오염 문제 부각, Python 편향 여전히 20%p+
 - 2025** ● **rebench / Pro** — 오염 대응, 장기 과제
→ 한계: 자동화-품질 균형 (31% 설치 성공률), Private 코드 접근 제한

맺음말

1

현재 벤치마크 숫자만으로 **SW 엔지니어링 능력**을 판단하기는 어렵다

— SWEBOK 18개 영역 중 코딩 일부만 평가, 오염 가능성 상존

2

다양한 언어와 환경을 포괄하면서도 **오염에 강건한** 벤치마크가 필요하다

— Python 편향 20%p+(Multi-SWE-bench), SWE-bench Pro의 Public vs Private 격차가 그 필요성을 실증

3

자동화와 품질 사이의 적정 지점을 찾는 것이 핵심 설계 과제다

— 낮은 수율 (rebench) vs 비공개 코드 접근 제한(Pro)

4

에이전틱 코딩 시대에 맞는 새로운 평가 방식 설계가 시작되고 있다

— 멀티 에이전트 코딩, 장기 태스크, 대화형 디버깅을 포함하는 방향으로

디스커션: 앞으로의 방향?

열린 질문들

- 어떤 벤치마크가 SE 역량을 가장 잘 반영할까?
- 오염을 완전히 막는 것은 가능한가?
- 에이전틱 코딩 시대의 평가는 어떻게 달라져야 하나?

여러분의 생각은?

지금 사용하시는 코딩 도구를 어떻게 평가하시나요?
벤치마크 점수가 그 판단에 얼마나 영향을 미치나요?

감사합니다

참고문헌

1. Chen et al., "Evaluating Large Language Models Trained on Code" (HumanEval), arXiv:2107.03374, 2021
2. Kulal et al., "SPoC: Search-based Pseudocode to Code," NeurIPS 2019
3. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", ICLR 2024, arXiv:2310.06770
4. OpenAI Preparedness Team, "Introducing SWE-bench Verified", 2024
5. Zan et al., "Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving," NeurIPS 2025, arXiv:2504.02605
6. Khandpur et al., "SWE-bench Multilingual", 2025
7. Liang et al., "The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason," NeurIPS 2025, arXiv:2506.12286
8. Badertdinov et al., "SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation," NeurIPS 2025, arXiv:2505.20411
9. Zhang et al., "SWE-bench Goes Live!," NeurIPS 2025, arXiv:2505.23419
10. Deng et al., "SWE-Bench Pro: Can AI Agents Solve Long-Horizon SE Tasks?", arXiv:2509.16941, 2025
11. Bourque & Fairley (eds.), "SWEBOK V4", IEEE, 2024
12. Hu et al., "Assessing and Advancing Benchmarks for Evaluating LLMs in SE Tasks," arXiv:2505.08903, 2025
13. Mundler et al., "SWT-Bench: Assessing Capabilities at Unit Test Generation," NeurIPS 2024
14. Wang et al., "SDLC Perspective: A Survey of Benchmarks for Code LLMs and Agents," arXiv:2505.05283, 2025
15. Xia, Deng, Zhang, "EvoEval: Evolving Coding Benchmarks via LLM," COLM 2024, arXiv:2403.19114